

(报告封面)

合肥工业大学

“数据结构”

课程设计报告

设计题目 求有向图简单路径

姓 名 李泽浩

学 号 2025218895

专 业 软件工程

班 级 软件工程 25-9 班

完成日期 2026.6.20

“数据结构课程设计”项目验收细则		
成绩等级	具体表现	教师评分
[90-100]	程序运行正确，求解问题技术思路表达清晰，回答问题清晰准确。	
[80-90)	程序运行正确，求解问题技术思路表达较清晰，回答问题较清楚准确。	
[70-80)	程序运行正确，求解问题技术思路表达基本清楚，问题回答大部分基本清楚准确。	
[60-70)	程序运行基本正确，求解问题技术思路表达尚清楚，个别问题回答清楚准确。	
[0-60)	程序运行出错，求解问题技术思路表达不清晰，问题回答不够清楚准确。	

“数据结构课程设计”报告评分细则		
成绩等级	具体表现	教师评分
[90-100]	◆ 逻辑结构合理，设计方案翔实准确，相关算法/技术描述清楚，报告内容翔实，系统展示结果正确； ◆ 文档格式规范，语句表达通顺；	
[80-90)	◆ 逻辑结构较合理，设计方案较翔实准确，相关算法/技术描述较清楚，报告内容较翔实，系统展示结果较正确； ◆ 文档格式较规范，语句表达较通顺；	
[70-80)	◆ 逻辑结构基本合理，设计方案尚翔实，相关算法/技术描述基本清楚，报告内容尚可，有系统展示结果； ◆ 文档格式尚规范，语句表达基本通顺；	
[60-70)	◆ 逻辑结构尚合理，设计方案欠翔实，相关算法/技术描述欠清楚，报告内容偏少，有少许系统展示； ◆ 文档格式欠规范，语句表达基本通顺；	
[0-60)	◆ 逻辑结构不合理，无设计方案、相关算法/技术描述不清楚，报告内容较少，无系统展示； ◆ 文档格式不规范，语句表达不通顺；	

教师签名：

时间：

1 题目及要求

题目：求有向图简单路径 **简单路径：** 如果一条路径上的顶点除了起点和终点可以相同外，其它顶点均不相同，则称此路径为一条简单路径；起点和终点相同的简单路径称为回路（或环）。

具体要求：

1. 求出有向图中从起点到终点的所有简单路径。其中起点和终点可以由用户自行设定。
2. 求出有向图中从起点到终点的指定长度（如 K ）的所有简单路径。其中起点和终点可以由用户自行设定。

2 需求分析与设计

2.1 需求分析

本课题要求在一个给定的有向图中，寻找两个指定顶点之间的简单路径。程序需要满足以下几点核心需求：

1. **数据读取能力：** 图的结构不能在代码里写死，需要能够从本地 `txt` 文件中动态读取顶点和边的数据。
2. **防死循环机制：** 有向图中极易出现环路（如 $A \rightarrow B \rightarrow C \rightarrow A$ ），程序必须有严格的访问标记机制，防止陷入死循环而导致栈溢出。
3. **筛选过滤能力：** 能够精确计算路径经过的边数，并在输出时过滤出符合长度 K 的路径。
4. **友好的交互能力：** 需要提供一个循环菜单供测试使用，且当用户输入不存在的顶点或非法的操作指令时，程序不能直接崩溃，应提示重新输入。

2.2 系统设计

- **数据结构选择：** 采用邻接表（Adjacency List）结合 `vector` 容器来存储图。相比于邻接矩阵，邻接表在遍历某个节点的所有后继节点时效率更高，非常适合做深度优先搜索（DFS）。
- **架构设计：** 采用模块化设计思想，将底层数据结构定义（`grpTypeDef.h`）、文件读取加载（`createGrpAdjLinkedList.h`）和核心寻路算法及测试主程序（`Keshe.cpp`）进行分离，降低代码的耦合度。

3 系统实现与使用方法

3.1 系统开发环境

- 操作系统： Windows
- 开发工具： Visual Studio Code (VS Code)
- 编译环境： g++ (GCC 编译器), C++11 及以上标准

3.2 系统界面简介

本系统采用终端命令行（控制台）交互界面，侧重于底层算法逻辑的验证。界面设计了清晰的边界符，操作提示以及步骤引导。系统启动时会一次性自动加载所有测试数据集，进入主菜单后，用户可通过输入数字编号在不同功能间自由切换，支持无限次循环测试。

3.3 系统功能模块简介

- 初始化与加载模块： 解析 test1.txt 等文件，过滤空行和注释，将文本转化为内存中的邻接表结构。
- 核心寻路模块： 包含 setGraphpath 函数，利用递归实现带有状态记录的图遍历。
- 指定长度过滤模块： 包装在 printPathByLength 函数中，通过判断动态数组的大小来精确限制路径的边数。
- 交互控制模块： 包含 while(true) 状态机和非法输入清空机制 (cin.clear()), 保障系统稳定运行。

3.4 重点算法介绍

本系统的核心难点在于有向图的遍历、路径状态的维护以及结果的精准筛选。系统将第一问的寻路逻辑封装在底层的 setGraphpath 函数中，并将第二问的过滤逻辑交由 printPathByLength 等函数处理。以下是这两个核心函数的具体实现剖析：

1. 核心回溯寻路： setGraphpath 函数

该函数是实现带有回溯的深度优先搜索（DFS）的核心引擎。其核心任务是找出从当前节点 `v` 到终点 `end` 的所有不重复路径，并将合法结果存入 `allpath` 集合中。具体执行步骤与代码逻辑如下：

- 状态记录：函数一进入，立即执行 `visited[v] = 1` 和 `path.push_back(v)`，将当前节点标记为已访问，并压入当前路径 `path` 中。这一步是防止图遍历陷入死循环的关键保障。
- 终点判定：随后判断当前节点是否到达目标（`v == end`）。如果到达，说明 `path` 中完整记录了一条有效路径，执行 `allpath.push_back(path)` 进行保存。收集完毕后，必须立即执行 `visited[v] = 0` 和 `path.pop_back()` 并 `return`，以便退回上一步，去寻找通往该终点的其它可能路径。
- 邻接表深度遍历：若当前节点不是终点，则通过指针 `EdgeNode* eR = G.VerList[v].firstEdge` 和 `eR->next`，遍历当前节点的所有出边。
- 条件递归与防环判断：在 `while(eR)` 循环中，程序会严格检查每一个邻接点 `eR->adjVer`。只有当该邻接点未被访问过（`!visited[eR->adjVer]`）时，才会向深层递归调用 `setGraphpath`。
- 回溯：当 `while` 循环结束，意味着当前节点 `v` 的所有向下分支都已探索完毕。此时准备退回上一层函数，必须在此处再次执行 `visited[v] = 0` 和 `path.pop_back()`。这是整个算法中最核心的防漏解机制，它抹除了当前节点的访问痕迹，使得该节点在后续其它分支的探索中依然是“可用”状态。

2. 路径长度 K 过滤：printPathByLength 函数

该函数负责实现课设要求中的第 ② 项功能，即“求出指定长度 `K` 的所有简单路径”。为了保持底层搜索算法的复用性，本函数采用“先全量获取，后精准过滤”的设计模式：

- 环境初始化与全量搜索：函数内部首先初始化一个局部的 `visited` 数组和存放结果的 `allpath` 容器，然后直接调用上述的 `setGraphpath` 引擎，一次性获取起点到终点的所有简单路径。
- 长度换算与精准匹配：获取到所有路径后，遍历 `allpath` 集合。数组中记录的是经过的顶点序列，而用户要求的路径长度 `K` 通常指边的数量。因此，判断条件被严格设定为 `(int)allpath[i].size() - 1 == K`。只有满足此换算公式的路径，才会被判定为符合要求的定长路径。
- 格式化输出与边界处理：匹配成功的路径会被遍历打印，并通过引用参数 `count` 进行计数。如果最终 `count` 依然为 0，程序会输出“（无）”，避免出现空白界面的交互。这种设计既满足了功能需求，又使得代码结构清晰、职责分明。

3.5 使用手册

- 确保可执行文件与三个测试数据文件（test1.txt, test2.txt, test3.txt）处于同一目录下。
- 运行程序，终端会提示初始化成功。
- 根据终端菜单提示，输入 1 选择查询所有路径，或输入 2 选择查询定长路径。
- 按照引导，输入要测试的图编号（1-3）以及起点和终点字母（如 A G）。
- 屏幕将打印出完整的路径序列及总条数。输入 0 可安全退出系统。

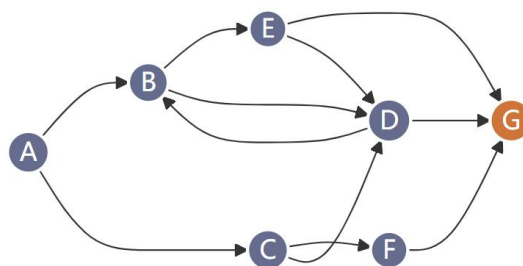
4 运行实例与系统功能测试

4.1 系统功能测试

1. 提供测试数据

1. 测试集 1 (test1.txt): 基础连通性与局部防环测试

test1



图的特点：

整体呈现从左向右的有向无环流向（A 到 G），但内部隐蔽地包含了一条局部逆向边（D → B）。

测试作用：

- 基础寻路验证：测试 DFS 能否正确纵向深入并找到基础的简单路径。
- 防止局部死循环验证： 检验 visited 数组是否能有效拦截 D → B →

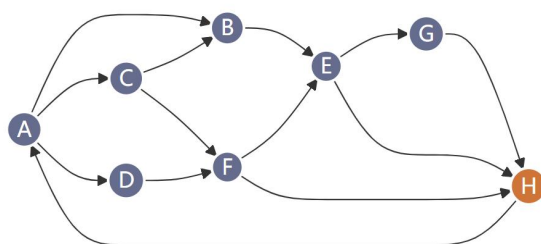
D 的无限循环。

确保的结果：

- 程序能够正常输出从起点到终点的多条路径。
- 当算法在节点 D 尝试走向 B 时，能够被 `visited[B] == 1` 成功阻挡并回溯，确保程序不会因为局部环路而引发栈溢出（Stack Overflow）崩溃。

2. 测试集 2 (test2.txt)：全局大回环与深度回溯极限测试

test2



图的特点：

结构紧凑，不仅存在多条不同的复杂路径如（ $A \rightarrow B, A \rightarrow C \rightarrow B$ ），最核心的是存在一条全局大回环边（ $H \rightarrow A$ ），将终点区域直接连回了起点区域。

测试作用：

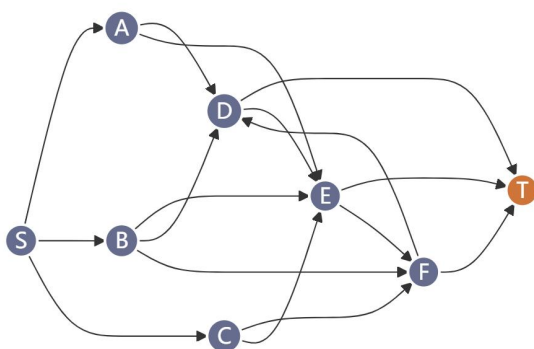
- 回溯状态校验：由于图谱高度交错，算法会频繁地在各个节点间穿梭、碰壁并退回。该图专门用于测试 `visited[v] = 0` 这一行恢复现场的代码是否严谨。
- 防漏解测试：在处理这类含有大回环的图时，稍有不慎就会因为提前封锁了节点而漏掉其他合法分支。

确保的结果：

- 证明算法在极度复杂的网络中依然能找出所有简单路径，绝对不会漏掉任何一个微小的有效解。
- 即使存在 $H \rightarrow A$ 这种大回环，只要 A 已经被标记为路径的起点（`visited[A] == 1`），程序就能在其试图绕回 A 时精准掐断，提取出真正的“简单路径”（即除起点和终点外，其余顶点不重复）。

3. 测试集 3 (test3.txt): 定长剪枝与高密度层次测试

test3



图的特点：

典型的分层交汇网络。起点 S 发射出三条线，中间经历了 A/B/C 层和 D/E/F 层的极高密度交叉融合（包含同层逆向边 $F \rightarrow D$ ），最后全部汇入终点 T。

测试作用：

- 指定长度 K 的功能专项测试：因为该图层次分明，从 S 到 T 存在大量长度相等或相近的组合路径，非常适合测试课设要求的第 ② 项功能。
- 剪枝效率校验：测试算法是否在路径长度大于 K 时及时停止向下搜索，从而节省算力。

确保的结果：

- 在使用“寻找长度为 K 的路径”功能时，能够精准无误地筛选出特定边数的路径。
- 验证算法具备处理“高密度处理”的能力。例如输入 $K = 3$ 时，程序能够在一堆复杂的网状连线中，清晰地剥离出类似 $S \rightarrow A \rightarrow D \rightarrow T$ 或 $S \rightarrow C \rightarrow F \rightarrow T$ 这种长度恰好为 3 的干净路径。

4.2 运行实例

为了充分验证系统的功能与鲁棒性，以下展示了三个具有代表性的运行实例，分别对基础路径、大回环图以及指定长度路径进行了测试。

实例 1：基础连通性与局部路径查询（基于 test1.txt）

测试目的： 验证系统对普通有向图的全部简单路径搜索能力。

操作过程： 选择功能 1，图 1，输入起点 A 和终点 G。

```
===== 系统初始化 =====
[*] 成功创建图 test1.txt!
[*] 成功创建图 test2.txt!
[*] 成功创建图 test3.txt!
=====

===== 简单路径求解系统 =====
1. 求解起点到终点的【所有简单路径】
2. 求解起点到终点的【指定长度简单路径】
0. 退出系统
=====

请输入功能序号 (0-2): 1

请选择要操作的图编号 (1-3): 1
请输入起点和终点 (用空格隔开, 如 A G): A G

----- 运行结果 -----
从顶点 A 到顶点 G 的所有简单路径 (共 6 条):
路径 1: A B D G
路径 2: A B E D G
路径 3: A B E G
路径 4: A C D G
路径 5: A C D B E G
路径 6: A C F G
-----
```

实例 2：大回环复杂网络的极限测试（基于 test2.txt）

测试目的： 验证系统在面对包含全局大回环（H → A）的高密度图时，回溯机制是否严谨，是否会漏解或陷入死循环。

操作过程： 选择功能 1，图 2，输入起点 A 和终点 H。

```
===== 简单路径求解系统 =====
1. 求解起点到终点的【所有简单路径】
2. 求解起点到终点的【指定长度简单路径】
0. 退出系统
=====

请输入功能序号 (0-2): 1

请选择要操作的图编号 (1-3): 2
请输入起点和终点 (用空格隔开, 如 A G): A H

----- 运行结果 -----
从顶点 A 到顶点 H 的所有简单路径 (共 10 条):
路径 1: A B E G H
路径 2: A B E H
路径 3: A C B E G H
路径 4: A C B E H
路径 5: A C F E G H
路径 6: A C F E H
路径 7: A C F H
路径 8: A D F E G H
路径 9: A D F E H
路径 10: A D F H
-----
```

实例 3：定长路径的精准剪枝与提取（基于 test3.txt）

测试目的： 验证系统求解指定长度（K）简单路径的功能。

操作过程： 选择功能 2，图 3，输入起点 S 和终点 T，指定长度 K 为 3（即路径包含 3 条边）。

```
===== 简单路径求解系统 =====
1. 求解起点到终点的【所有简单路径】
2. 求解起点到终点的【指定长度简单路径】
0. 退出系统
=====
请输入功能序号 (0-2): 2

请选择要操作的图编号 (1-3): 3
请输入起点和终点 (用空格隔开, 如 A G): S T

----- 运行结果 -----
请输入指定路径长度 K (边数): 2
从顶点 S 到顶点 T 的长度为 2 的简单路径:
(无)
-----
```

```
===== 简单路径求解系统 =====
1. 求解起点到终点的【所有简单路径】
2. 求解起点到终点的【指定长度简单路径】
0. 退出系统
=====
请输入功能序号 (0-2): 2

请选择要操作的图编号 (1-3): 3
请输入起点和终点 (用空格隔开, 如 A G): S T

----- 运行结果 -----
请输入指定路径长度 K (边数): 3
从顶点 S 到顶点 T 的长度为 3 的简单路径:
路径 1: S A D T
路径 2: S A E T
路径 3: S B D T
路径 4: S B F T
路径 5: S C E T
路径 6: S C F T
共 6 条
-----
```

结束程序

```
===== 简单路径求解系统 =====
1. 求解起点到终点的【所有简单路径】
2. 求解起点到终点的【指定长度简单路径】
0. 退出系统
=====
请输入功能序号 (0-2): 0

[*] 感谢使用, 系统正在退出...
```

5 总结与进一步改进

5.1 总结（心得体会）

作为一名大一下的软件工程学生，本次“求有向图简单路径”的课程设计是我第一次真正意义上从零开始，独立完成一个包含多模块拆分、文件动态加载、复杂数据结构构建以及核心递归算法实现的完整 C++工程项目。回首整个开发周期，虽然充满了无数次调试与报错的挑战，但当看到程序在控制台中稳健运行的那一刻，我感到收获极其巨大。

1. 算法逻辑与底层排错的深刻顿悟

在算法设计初期，我天真地以为只要照搬书本上的 DFS 模板就能轻松解决问题。然而，在进行第一轮测试时，面对测试集 2 中错综复杂的“全局大回环（H → A）”结构，程序频繁陷入死循环，甚至直接抛出“Stack Overflow”或异常崩溃。为了解决定位问题，我放弃了盲目修改代码，而是静下心来在草稿纸上完整地画出了算法运行时的“递归状态空间树”。通过模拟代码的压栈与弹栈过程，我深刻认识到了遍历中一个极其容易被忽视的致命陷阱，由于回溯时状态重置不彻底，算法在提前封锁了某些节点后，不仅会漏掉大量合法的分支解，更会在面对回环

时彻底迷失方向。在代码的退出递归层处写下 `visited[v] = 0` 和 `path.pop_back()`; 这两行回溯代码后, 程序最终能够顺利运行。那一刻的成就是无法用言语表达的, 它让我真正体会到了数据结构与算法设计的严谨与精妙魅力。

2. 模块化的思维

除了核心算法的突破, 本次课设最大的收获在于让我初步建立了模块化的思维。以前写作业, 我习惯于把所有代码堆叠在一个 `main.cpp` 文件里。但在这次项目中, 我尝试将底层的邻接表/邻接矩阵定义 (`grpTypeDef.h`)、文件读取功能 (`createGrpAdjLinkedList.h`) 以及数据文件进行拆分模块化。这不仅让我在后续排查错误时思路更加清晰, 更使得代码的复用性得到了提升。例如, 当我需要增加对输入参数的容错处理时(如使用 `cin.clear()` 防止用户误输英文字母导致的死循环), 我只需要在主测试模块中修改, 而完全不用担心会破坏底层的寻路算法。这个过程极大地提升了我的模块化思维。

3. 结语

本次《数据结构》课程设计不仅是一次代码能力的练兵, 更是一场思维逻辑的锻炼。它让我从理论走向了实践, 让我明白了一个优秀的软件不仅仅是能算出结果, 更是要在不同的数据下保持健壮, 在代码结构上清晰。这为我未来的专业学习打下了一针强心剂, 也让我对后续更复杂的开发充满了期待。

5.2 进一步改进及建议

- **性能优化:** 目前寻找指定长度 `K` 的路径时, 是在先搜出所有路径的基础上进行过滤。未来可以考虑直接在 `DFS` 参数中加入 `current_length`, 当深度大于 `K` 时直接 `return` (剪枝), 从而进一步节省系统开销。
- **界面升级:** 当前为控制台纯文本交互模式。随着后续编程语言的学习, 可以考虑引入轻量级的图形化界面库, 让用户通过鼠标点击就能高亮显示走过的路径, 增强系统的可视化。

6 附录—源程序

源程序分为 3 个库文件, 1 个主程序, 三个测试数据集

库文件 1-`grpTypeDef.h`

```
#ifndef GRPTYPEDEF_H
#define GRPTYPEDEF_H

#include <cstdio>
#include <cstring>
#include <cstdlib>
using namespace std;
#define MAXVEX 100 //最大顶点数
```

```

#define INF 65535      //无穷大
//图类型枚举
typedef enum {
    UDG,    //无向图
    UDN,    //无向网
    DG,     //有向图
    DN      //有向网
} GraphKind;
typedef char elementType; //顶点元素类型
typedef int eInfoType;    //边信息类型（权值）
typedef int cellType;     //邻接矩阵单元格类型
//邻接表边结点
typedef struct EdgeNode {
    int adjVer;           //邻接点编号
    eInfoType eInfo;      //边信息
    struct EdgeNode* next; //下一条边
} EdgeNode;
//邻接表顶点结点
typedef struct {
    elementType data;      //顶点数据
    EdgeNode* firstEdge;   //第一条边
} VexNode;
//图结构体（同时支持邻接矩阵和邻接表两种存储方式）
typedef struct {
    elementType Data[MAXVEX + 1]; //顶点数据（邻接矩阵用）
    cellType AdjMatrix[MAXVEX + 1][MAXVEX + 1]; //邻接矩阵
    VexNode VerList[MAXVEX + 1]; //邻接表顶点数组
    int VerNum; //顶点数
    int ArcNum; //边数
    GraphKind gKind; //图类型
} Graph;
//删除字符串左边空格
void strLTrim(char* str);
//销毁图（邻接表用）
void DestroyGraph(Graph &G);
#endif

```

库文件 2-createGrpAdjLinkedList.h

```

#ifndef CREATEGRPADJLINKEDLIST_H
#define CREATEGRPADJLINKEDLIST_H

#include "grpTypeDef.h"

//*****从数据文件创建图*****//

```

```

/* 函数功能：从文本文件创建邻接表表示的图                                */
/* 入口参数  char fileName[], 文件名                                    */
/* 出口参数: Graph &G, 创建的图                                        */
/* 返回值: bool, true 创建成功; false 创建失败                        */
/* 函数名: CreateGrpFromFile1(char fileName[], Graph &G)            */
/* 备注   : 该函数为旧版实现, 直接解析邻接矩阵格式数据文件        */
/******//
bool CreateGrpFromFile1(char fileName[], Graph &G)
{
    FILE* pFile;    //定义顺序表的文件指针
    char str[1000]; //存放读出一行文本的字符串
    char strTemp[10]; //判断是否注释行
    char* ss; //字符串指针, 指向 fgets 函数返回的字符串地址
    int i=0,j=0;
    int edgeNum=0; //边的数量
    GraphKind graphType; //图类型枚举变量
    pFile=fopen(fileName,"r");
    if(!pFile)
    {
        printf("错误: 文件%s 打开失败.\n",fileName);
        return false;
    }

    ss=fgets(str,1000,pFile);//读取第一行数据
    strncpy(strTemp,str,2);
    while((ss!=NULL) && (strstr(strTemp,"//")!=NULL) ) //跳过注释行
    {
        ss=fgets(str,1000,pFile);
        strncpy(strTemp,str,2);
        //cout<<strTemp<<endl;
    }

    //循环结束, str 中应该已经是文件标识, 判断文件格式
    //cout<<str<<endl;
    if(strstr(str,"Graph")==NULL)
    {
        printf("错误: 打开的文件格式错误! \n");
        fclose(pFile); //关闭文件
        return false;
    }
    //读取图的类型
    if(fgets(str,1000,pFile)==NULL)
    {
        printf("错误: 读取图的类型标记失败! \n");
        fclose(pFile); //关闭文件
    }
}

```

```

        return false;
    }
    //设置图的类型
    if(strstr(str,"UDG"))
        graphType=UDG; //无向图
    else if(strstr(str,"UDN"))
        graphType=UDN; //无向网
    else if(strstr(str,"DG"))
        graphType=DG; //有向图
    else if(strstr(str,"DN"))
        graphType=DN; //有向网
    else
    {
        printf("错误: 读取图的类型标记失败! \n");
        fclose(pFile); //关闭文件
        return false;
    }
    //读取顶点元素到 str
    if(fgets(str,1000,pFile)==NULL)
    {
        printf("错误: 读取图的顶点数据失败! \n");
        fclose(pFile); //关闭文件
        return false;
    }
    //顶点数据放入图的顶点数组
    char* token=strtok(str, " ");
    int nNum=1; //顶点数
    while(token!=NULL)
    {
        G.VerList[nNum].data=*token;//顶点数据转换为整数，若为字符则不需转
        G.VerList[nNum].firstEdge=NULL;
        //p=NULL;
        //eR=G.VerList[i].firstEdge;
        token = strtok( NULL, " ");
        nNum++;
    }

    //图的邻接矩阵数据转换为邻接表
    int nRow=1; //矩阵行下标，从 1 开始
    int nCol=1; //矩阵列下标，从 1 开始
    EdgeNode* eR; //边链表尾指针
    EdgeNode* p; //边结点指针

```

换

```

while(fgets(str,1000,pFile)!=NULL)
{
    eR=NULL;
    p=NULL;
    nCol=1; //列下标从 1 开始
    char* token=strtok(str," "); //以空格为分隔符，分割一行数据，写入邻接表
    while(token!=NULL)
    {
        if(atoi(token)>=1 && atoi(token)<INF) //有边存在
        {
            p=new EdgeNode; //创建一个邻接点边结点
            p->adjVer=nCol; //邻接点编号为列号
            p->eInfo=atoi(token); //取得边的附加信息
            p->next=NULL;
            if(G.VerList[nRow].firstEdge==NULL)
            {
                G.VerList[nRow].firstEdge=p;
                eR=p;
            }
            else
            {
                eR->next=p;
                eR=p; //尾指针后移
            }
            edgeNum++; //边数加 1
        }
        token = strtok( NULL, " "); //读取下一个子串
        nCol++; //列号加 1
    }
    nRow++; //行号加 1，读取下一行
}
G.VerNum=nNum; //图的顶点数
if(graphType==UDG || graphType==UDN)
    G.ArcNum=edgeNum / 2; //无向图或网的边数等于统计的数字除 2
else
    G.ArcNum=edgeNum;
G.gKind=graphType; //图的类型
fclose(pFile); //关闭文件
return true;
}

//*****从数据文件创建图*****//
/* 函数功能：从文本文件创建邻接表表示的图 */
/* 入口参数 char fileName[], 文件名 */
/* 出口参数：Graph &G, 创建的图 */

```

```

/* 返回值: bool, true 创建成功; false 创建失败 */
/* 函数名: CreateGraphFromFile(char fileName[], Graph &G) */
/* 备注: 该函数为改进版实现, 支持跳过空行和注释行 */
/*****//
bool CreateGraphFromFile(char fileName[], Graph &G)
{
    FILE* pFile;          //定义顺序表的文件指针
    char str[1000];        //存放读出一行文本的字符串
    char strTemp[10];      //判断是否注释行
    int i=0,j=0;
    int edgeNum=0;         //边的数量
    eInfoType eWeight;     //边的信息, 常为边的权值
    GraphKind graphType;   //图类型枚举变量
    pFile=fopen(fileName,"r");
    if(!pFile)
    {
        printf("错误: 文件%s 打开失败.\n",fileName);
        return false;
    }

    while(fgets(str,1000,pFile)!=NULL) //跳过空行和注释行
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\n') //空行, 继续读取下一行
            continue;
        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //跳过注释行
            continue;
        else //非注释行、非空行, 跳出循环
            break;
    }

    //循环结束, str 中应该已经是文件标识, 判断文件格式
    if(strstr(str,"Graph")==NULL)
    {
        printf("错误: 打开的文件格式错误! \n");
        fclose(pFile); //关闭文件
        return false;
    }

    //读取图的类型, 跳过空行
    while(fgets(str,1000,pFile)!=NULL)
    {
        //删除字符串左边空格
        strLTrim(str);

```

```

        if (str[0]=='\n') //空行，继续读取下一行
            continue;
        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //注释行，跳过，继续读取下一行
            continue;
        else //非空行，也非注释行，即图的类型标识
            break;
    }

    //设置图的类型
    if(strstr(str,"UDG"))
        graphType=UDG; //无向图
    else if(strstr(str,"UDN"))
        graphType=UDN; //无向网
    else if(strstr(str,"DG"))
        graphType=DG; //有向图
    else if(strstr(str,"DN"))
        graphType=DN; //有向网
    else
    {
        printf("错误: 读取图的类型标记失败! \n");
        fclose(pFile); //关闭文件
        return false;
    }

    //读取顶点元素，到 str，跳过空行
    while(fgets(str,1000,pFile)!=NULL)
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\n') //空行，继续读取下一行
            continue;
        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //注释行，跳过，继续读取下一行
            continue;
        else //非空行，也非注释行，即图的顶点元素行
            break;
    }

    //顶点数据放入图的顶点数组
    char* token=strtok(str," ");
    int nNum=0;
    while(token!=NULL)
    {
        G.VerList[nNum+1].data=*token;
        G.VerList[nNum+1].firstEdge=NULL;
        //p=NULL;
    }

```

```

        //eR=G.VerList[i].firstEdge;
        token = strtok( NULL, " ");
        nNum++;
    } //顶点数为 nNum, 编号从 1 到 nNum

    //循环读取边的数据到邻接表
    int nRow=1; //矩阵行下标, 从 1 开始
    int nCol=1; //矩阵列下标, 从 1 开始
    EdgeNode* eR; //边链表尾指针
    EdgeNode* p;
    elementType Nf,Ns; //边或弧的 2 个相邻顶点
    while(fgets(str,1000,pFile)!=NULL)
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\n') //空行, 继续读取下一行
            continue;

        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //注释行, 跳过, 继续读取下一行
            continue;
        //nCol=0; //列下标从 0 开始
        char* token=strtok(str," "); //以空格为分隔符, 分割一行数据, 写入邻接表

        if(token==NULL) //分割为空串, 失败退出
        {
            printf("错误: 读取图的边数据失败! \n");
            fclose(pFile); //关闭文件
            return false;
        }
        Nf=*token; //获取边的第一个顶点

        token = strtok( NULL, " "); //读取下一个子串, 即第二个顶点
        if(token==NULL) //分割为空串, 失败退出
        {
            printf("错误: 读取图的边数据失败! \n");
            fclose(pFile); //关闭文件
            return false;
        }
        Ns=*token; //获取边的第二个顶点
        //从第一个顶点获取行号
        for(nRow=1;nRow<=nNum;nRow++)
        {
            if(G.VerList[nRow].data==Nf) //从顶点列表找到第一个顶点的编号

```

```

        break;
    }
    //从第二个顶点获取列号
    for(nCol=1;nCol<=nNum;nCol++)
    {
        if(G.VerList[nCol].data==Ns) //从顶点列表找到第二个顶点的编号
            break;
    }
    //如果为网，读取权值
    if(graphType==UDN || graphType==DN)
    {
        token = strtok( NULL, " "); //读取下一个子串，即边的附加信息，常为边的权重
        if(token==NULL) //分割为空串，失败退出
        {
            printf("错误：读取图的边数据失败！\n");
            fclose(pFile); //关闭文件
            return false;
        }
        eWeight=atoi(token); //取得边的附加信息
    }
    eR=G.VerList[nRow].firstEdge;
    while(eR!=NULL && eR->next!=NULL)
    {
        eR=eR->next; //移动到链表尾，找到尾结点
    }

    p=new EdgeNode; //创建一个邻接点边结点
    p->adjVer=nCol; //邻接点编号为列号
    if(graphType==UDN || graphType==DN) //如果为网，邻接表中对应的边设置权值，否则置
为 1
        p->eInfo=eWeight;
    else
        p->eInfo=1;
    p->next=NULL;

    if(G.VerList[nRow].firstEdge==NULL)
    {
        G.VerList[nRow].firstEdge=p;
        eR=p;
    }
    else
    {
        eR->next=p;
        eR=p; //尾指针后移
    }

```

```

    }
    edgeNum++;    //边数加 1
}

G.VerNum=nNum;    //图的顶点数
if(graphType==UDG || graphType==UDN)
    G.ArcNum=edgeNum / 2;    //无向图或网的边数等于统计的数字除 2
else
    G.ArcNum=edgeNum;
G.gKind=graphType;    //图的类型
fclose(pFile);    //关闭文件
return true;
}
//销毁图
void DestroyGraph(Graph &G)
{
    EdgeNode *p,*u;
    int vID;
    for(vID=1; vID<=G.VerNum; vID++)    //对每个顶点
    {
        p=G.VerList[vID].firstEdge;
        G.VerList[vID].firstEdge=NULL;
        while(p)    //删除链表中的所有边结点
        {
            u=p->next;    //u 指向下一个结点
            delete(p);    //删除当前结点
            p=u;
        }
    }
    p=NULL;
    u=NULL;
    G.VerNum=-1;    //顶点数置为无效
}
#endif

```

库文件 2-createGrpAdjMatrix.h

```

#ifndef CREATEGRPADJMATRIX_H
#define CREATEGRPADJMATRIX_H

#include "grpTypeDef.h"

/*****从数据文件创建图*****/
/* 函数功能: 从文本文件创建邻接矩阵表示的图 */
/* 入口参数 char fileName[], 文件名 */
/* 出口参数: Graph &G, 创建的图 */

```

```

/* 返回值: bool, true 创建成功; false 创建失败 */
/* 函数名: CreateGrpFromFile(char fileName[], Graph &G) */
/*****//
bool CreateGrpFromFile(char fileName[], Graph &G)
{
    FILE* pFile;        //定义顺序表的文件指针
    char str[1000];      //存放读出一行文本的字符串
    char strTemp[10];    //判断是否注释行
    cellType eWeight;    //边的信息, 常为边的权值
    GraphKind GrpType;   //图类型枚举变量
    pFile=fopen(fileName,"r");
    if(!pFile)
    {
        printf("错误: 文件%s 打开失败。\\n",fileName);
        return false;
    }
    while(fgets(str,1000,pFile)!=NULL)
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\\n') //空行, 继续读取下一行
            continue;
        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //跳过注释行
            continue;
        else //非注释行、非空行, 跳出循环
            break;
    }
    //循环结束, str 中应该已经是文件标识, 判断文件格式
    if(strstr(str,"Graph")==NULL)
    {
        printf("错误: 打开的文件格式错误! \\n");
        fclose(pFile); //关闭文件
        return false;
    }
    //读取图的类型, 跳过空行
    while(fgets(str,1000,pFile)!=NULL)
    {
        //删除字符串左边空格
        strLTrim(str);
        if (str[0]=='\\n') //空行, 继续读取下一行
            continue;
        strncpy(strTemp,str,2);
        if(strstr(strTemp,"//")!=NULL) //注释行, 跳过, 继续读取下一行

```

```

        continue;
    else //非空行，也非注释行，即图的类型标识
        break;
}
//设置图的类型
if(strstr(str,"UDG"))
    GrpType=UDG; //无向图
else if(strstr(str,"UDN"))
    GrpType=UDN; //无向网
else if(strstr(str,"DG"))
    GrpType=DG; //有向图
else if(strstr(str,"DN"))
    GrpType=DN; //有向网
else
{
    printf("错误：读取图的类型标记失败！\n");
    fclose(pFile); //关闭文件
    return false;
}

```

```

//读取顶点元素，到 str。跳过空行
while(fgets(str,1000,pFile)!=NULL)
{
    //删除字符串左边空格
    strLTrim(str);
    if (str[0]=='\n') //空行，继续读取下一行
        continue;
    strncpy(strTemp,str,2);
    if(strstr(strTemp,"//")!=NULL) //注释行，跳过，继续读取下一行
        continue;
    else //非空行，也非注释行，即图的顶点元素行
        break;
}
//顶点数据放入图的顶点数组
char* token=strtok(str," ");//以空格为分隔符，分割一行数据，写入图的顶点数组
int nNum=1; //顶点数
while(token!=NULL)
{
    G.Data[nNum]=*token; // atoi(token); //顶点数据转换为整数，若为字符则不需转换
    token = strtok( NULL, " ");//继续分割下一个子串
    nNum++;
}
nNum--; //顶点数
//图的邻接矩阵初始化

```

```

int nRow=1; //矩阵行下标, 从 1 开始
int nCol=1; //矩阵列下标, 从 1 开始
if(GrpType==UDG || GrpType==DG)//如果为图, 邻接矩阵中对应的边设置为 1, 否则设置为 INF
(无穷大)
{
    for(nRow=1;nRow<=nNum;nRow++)
        for(nCol=1;nCol<=nNum;nCol++)
            G.AdjMatrix[nRow][nCol]=0;
}
else
{
    for(nRow=1;nRow<=nNum;nRow++)
        for(nCol=1;nCol<=nNum;nCol++)
            G.AdjMatrix[nRow][nCol]=INF; //INF 表示无穷大
}
//循环读取边的数据到邻接矩阵
int edgeNum=0; //边的数量
elementType Nf,Ns; //边或弧的 2 个相邻顶点
while(fgets(str,1000,pFile)!=NULL)
{
    //删除字符串左边空格
    strLTrim(str);
    if (str[0]=='\n') //空行, 继续读取下一行
        continue;
    strncpy(strTemp,str,2);
    if(strstr(strTemp,"//")!=NULL) //注释行, 跳过, 继续读取下一行
        continue;
    char* token=strtok(str," "); //以空格为分隔符, 分割一行数据, 写入邻接矩阵
    if(token==NULL) //分割为空串, 失败退出
    {
        printf("错误: 读取图的边数据失败! \n");
        fclose(pFile); //关闭文件
        return false;
    }
    Nf=*token; //获取边的第一个顶点
    token = strtok( NULL, " "); //读取下一个子串, 即第二个顶点
    if(token==NULL) //分割为空串, 失败退出
    {
        printf("错误: 读取图的边数据失败! \n");
        fclose(pFile); //关闭文件
        return false;
    }
    Ns=*token; //获取边的第二个顶点
    //从第一个顶点获取行号

```

```

for(nRow=1;nRow<=nNum;nRow++)
{
    if(G.Data[nRow]==Nf) //从顶点列表找到第一个顶点的编号
        break;
}
//从第二个顶点获取列号
for(nCol=1;nCol<=nNum;nCol++)
{
    if(G.Data[nCol]==Ns) //从顶点列表找到第二个顶点的编号
        break;
}

```

```

//如果为网，读取权值
if(GrpType==UDN || GrpType==DN)
{
    token = strtok( NULL, " "); //读取下一个子串，即边的附加信息，常为边的权重
    if(token==NULL) //分割为空串，失败退出
    {
        printf("错误：读取图的边数据失败！\n");
        fclose(pFile); //关闭文件
        return false;
    }
    eWeight=atoi(token); //取得边的附加信息
}
if(GrpType==UDN || GrpType==DN) //如果为网，邻接矩阵中对应的边设置权值，否则置为
1
    G.AdjMatrix[nRow][nCol]=eWeight;
else
    G.AdjMatrix[nRow][nCol]=1; //atoi(token); //字符串转为整数
edgeNum++; //边数加 1
}
G.VerNum=nNum; //图的顶点数
if(GrpType==UDG || GrpType==UDN)
    G.ArcNum=edgeNum / 2; //无向图或网的边数等于统计的数字除 2
else
    G.ArcNum=edgeNum; //有向图或网的边数等于统计的数字
G.gKind=GrpType; //图的类型
fclose(pFile); //关闭文件
return true;
}
//删除字符串、字符数组左边空格
void strLTrim(char* str)
{
    int i,j;

```

```

    int n=0;
    n=strlen(str)+1;
    for(i=0;i<n;i++)
    {
        if(str[i]!=' ') //找到左起第一个非空格位置
            break;
    }

    //以第一个非空格字符为首字符移动字符串
    for(j=0;j<n;j++)
    {
        str[j]=str[i];
        i++;
    }
}
#endif

```

主程序-Keshe.cpp

```

#include <iostream>
#include <vector>
#include "grpTypeDef.h"
#include "createGrpAdjMatrix.h"
#include "createGrpAdjLinkedList.h"
using namespace std;

// ===== 求所有简单路径 =====
void setGraphpath(Graph& G, int visited[], vector<int>& path, int& pathNum,
    int v, int end, vector<vector<int>>& allpath) {
    visited[v] = 1;
    path.push_back(v);
    if (v == end) {
        allpath.push_back(path);
        pathNum++;
        visited[v] = 0;
        path.pop_back();
        return;
    }
    EdgeNode* eR = G.VerList[v].firstEdge;
    while (eR) {
        if (!visited[eR->adjVer])
            setGraphpath(G, visited, path, pathNum, eR->adjVer, end, allpath);
        eR = eR->next;
    }
    visited[v] = 0;
    path.pop_back();
}

```

```

}
// 打印所有简单路径
void printPath(Graph& G, int start, int end) {
    int visited[MAXVEX + 1] = {0};
    vector<int> path;
    vector<vector<int>> allpath;
    int pathNum = 0;
    setGraphpath(G, visited, path, pathNum, start, end, allpath);
    cout << "从顶点 " << G.VerList[start].data
        << " 到顶点 " << G.VerList[end].data
        << " 的所有简单路径 (共 " << pathNum << " 条): " << endl;
    for (int i = 0; i < (int)allpath.size(); i++) {
        cout << "路径 " << i + 1 << ": ";
        for (int v : allpath[i]) cout << G.VerList[v].data << " ";
        cout << endl;
    }
}
// ===== 求指定长度 K 的简单路径 =====
void printPathByLength(Graph& G, int start, int end, int K, int& count) {
    int visited[MAXVEX + 1] = {0};
    vector<int> path;
    vector<vector<int>> allpath;
    int pathNum = 0;
    setGraphpath(G, visited, path, pathNum, start, end, allpath);
    count = 0;
    cout << "从顶点 " << G.VerList[start].data
        << " 到顶点 " << G.VerList[end].data
        << " 的长度为 " << K << " 的简单路径: " << endl;
    for (int i = 0; i < (int)allpath.size(); i++) {
        if ((int)allpath[i].size() - 1 == K) {
            cout << "路径 " << ++count << ": ";
            for (int v : allpath[i]) cout << G.VerList[v].data << " ";
            cout << endl;
        }
    }
    if (!count) cout << " (无) " << endl;
    else cout << "共 " << count << " 条" << endl;
}
// ===== 主函数 =====
int main() {
    // 1. 系统初始化: 一次性加载所有图
    Graph G[3] = {};
    char grpFile[3][20] = {"test1.txt", "test2.txt", "test3.txt"};

```

```

cout << "===== 系统初始化 =====" << endl;
for (int i = 0; i < 3; i++) {
    if (!CreateGraphFromFile(grpFile[i], G[i])) {
        cout << "[!] 创建图 " << grpFile[i] << " 失败! " << endl;
        return -1;
    }
    cout << "[*] 成功创建图 " << grpFile[i] << "!" << endl;
}
cout << "===== " << endl;
// 2. 交互式主循环：允许用户反复进行测试
while (true) {
    cout << "\n===== 简单路径求解系统 =====" << endl;
    cout << " 1. 求解起点到终点的【所有简单路径】" << endl;
    cout << " 2. 求解起点到终点的【指定长度简单路径】" << endl;
    cout << " 0. 退出系统" << endl;
    cout << "===== " << endl;
    cout << "请输入功能序号 (0-2): ";
    int choice;
    // 防御性编程：防止用户输入字母导致死循环
    if (!(cin >> choice)) {
        cin.clear();
        cin.ignore(10000, '\n');
        cout << "[!] 输入无效，请输入数字! " << endl;
        continue;
    }
    // 退出选项
    if (choice == 0) {
        cout << "\n[*] 感谢使用，系统正在退出..." << endl;
        break;
    }
    // 错误输入拦截
    if (choice != 1 && choice != 2) {
        cout << "[!] 无效的选择，请重新输入! " << endl;
        continue;
    }
    // 3. 分步引导用户输入参数
    int graphId;
    cout << "\n请选择要操作的图编号 (1-3): ";
    cin >> graphId;
    if (graphId < 1 || graphId > 3) {
        cout << "[!] 图编号不存在，已返回主菜单! " << endl;
        continue;
    }
    int gIndex = graphId - 1; // 转换为数组索引

```

```

char startChar, endChar;
cout << "请输入起点和终点（用空格隔开，如 A G）： ";
cin >> startChar >> endChar;
// 校验顶点是否存在
int start = 0, end = 0;
for (int i = 1; i <= G[gIndex].VerNum; i++) {
    if (G[gIndex].VerList[i].data == startChar) start = i;
    if (G[gIndex].VerList[i].data == endChar) end = i;
}
if (start == 0 || end == 0) {
    cout << "[!] 错误：起点或终点在图 " << graphId << " 中不存在！ " << endl;
    continue; // 容错处理：不退出程序，而是跳过本次循环，回到主菜单
}
// 4. 执行核心算法并展示结果
cout << "\n----- 运行结果 -----" << endl;
if (choice == 1) {
    printPath(G[gIndex], start, end);
} else if (choice == 2) {
    int K;
    cout << "请输入指定路径长度 K（边数）： ";
    cin >> K;
    int count = 0;
    printPathByLength(G[gIndex], start, end, K, count);
}
cout << "-----" << endl;
}
// 5. 释放资源
for (int i = 0; i < 3; i++) {
    DestroyGraph(G[i]);
}

return 0;
}

```

测试数据集 1-test1.txt

```

Graph
DG
A B C D E F G
A B
A C
B D
B E
C D
C F

```

```
D G
E D
E G
F G
D B
```

测试数据集 2-test2. txt

```
Graph
DG
A B C D E F G H
A B
A C
A D
B E
C B
C F
D F
E G
E H
F E
F H
G H
H A
```

测试数据集 3-test3. txt

```
Graph
DG
S A B C D E F T
S A
S B
S C
A D
A E
B D
B F
C E
C F
D E
D T
E T
F T
E F
F D
```